



Exp :05 Write the python program for Tic Tac Toe game

Aim: To write a Python program to implement a two-player Tic Tac Toe game using functions and lists.

Algorithm:

Step 1: Create a data structure to represent the state of the problem, including the number of missionaries and cannibals on each side of the river and the position of the boat.

Step 2: Create a data structure to represent the state space of the problem, including the initial state, the goal state, and the possible transitions between states.

Step 3: Use a search algorithm, such as depth-first search or breadth-first search, to explore the state space and find a solution.

Step 4: At each step of the search, generate all possible successor states from the current state by applying one of the allowed boat movements (two missionaries, two cannibals, or one of each).

Step 5: Check if the successor state is valid (i.e., no more cannibals than missionaries on either side of the river).

Step 6: If the successor state is valid, add it to the search tree and continue the search. Step 7: If the successor state is the goal state, return the path from the initial state to the goal state.

Step 8: If there are no more possible successor states to explore, backtrack to the previous state and try another possible movement.

Program:

```
from collections import deque
```

```
def is_valid(m, c):
```

```
    if m < 0 or c < 0 or m > 3 or c > 3:
```

```
        return False
```

```
    if m > 0 and m < c:
```

```
        return False
```

```
    if 3 - m > 0 and 3 - m < 3 - c:
```

```
    return False
```

```
    return True
```

```
def get_successors(state):
```

```
    m, c, b = state
```

```
    successors = []
```

```
    moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]
```

```
    for dm, dc in moves:
```

```
        if b == 1:
```

```
            nm, nc, nb = m - dm, c - dc, 0
```

```
        else:
```

```
            nm, nc, nb = m + dm, c + dc, 1
```

```
        if is_valid(nm, nc):
```

```
            successors.append((nm, nc, nb))
```

```
    return successors
```

```
def solve_missionaries_cannibals():
```

```
    initial_state = (3, 3, 1)
```

```
    goal_state = (0, 0, 0)
```

```
    queue = deque([(initial_state, [initial_state])])
```

```
    visited = {initial_state}
```

```
    while queue:
```

```
        current_state, path = queue.popleft()
```

```
        if current_state == goal_state:
```

```
            return path
```

```

for successor in get_successors(current_state):
    if successor not in visited:
        visited.add(successor)
        queue.append((successor, path + [successor]))

return None

if __name__ == '__main__':
    solution = solve_missionaries_cannibals()

    if solution:
        for step_num, step in enumerate(solution):
            m, c, b = step
            boat_pos = "Left" if b == 1 else "Right"
            print("Step", step_num, "- Left Side: M =", m, "C =", c, " | Boat:", boat_pos, " | Right
Side: M =", 3 - m, "C =", 3 - c)
        else:
            print("No solution found.")

```

OUTPUT:

```

Step 0 - Left Side: M = 3 C = 3 | Boat: Left | Right Side: M = 0 C = 0
Step 1 - Left Side: M = 3 C = 1 | Boat: Right | Right Side: M = 0 C = 2
Step 2 - Left Side: M = 3 C = 2 | Boat: Left | Right Side: M = 0 C = 1
Step 3 - Left Side: M = 3 C = 0 | Boat: Right | Right Side: M = 0 C = 3
Step 4 - Left Side: M = 3 C = 1 | Boat: Left | Right Side: M = 0 C = 2
Step 5 - Left Side: M = 1 C = 1 | Boat: Right | Right Side: M = 2 C = 2
Step 6 - Left Side: M = 2 C = 2 | Boat: Left | Right Side: M = 1 C = 1
Step 7 - Left Side: M = 0 C = 2 | Boat: Right | Right Side: M = 3 C = 1
Step 8 - Left Side: M = 0 C = 3 | Boat: Left | Right Side: M = 3 C = 0
Step 9 - Left Side: M = 0 C = 1 | Boat: Right | Right Side: M = 3 C = 2
Step 10 - Left Side: M = 1 C = 1 | Boat: Left | Right Side: M = 2 C = 2
Step 11 - Left Side: M = 0 C = 0 | Boat: Right | Right Side: M = 3 C = 3

```

Result: Thus, the program was executed and found that the output was found to be correct.